Program 4

Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.

```
from gensim.models import Word2Vec

# Training small local data (offline embeddings)

data = [

    ["natural","disaster","earthquake","flood","cyclone","tsunami"],

    ["earthquake","damage","buildings","people"],

    ["flood","rain","water","loss"],

    ["cyclone","storm","wind","destruction"]

]

model = Word2Vec(data, vector_size=50, min_count=1)

original = "write an essay on natural disaster"

# Get similar words using embeddings

similar = [w for w, s in model.wv.most_similar("disaster", topn=3)]

enriched = original + " including " + ", ".join(similar)

def generate(text):

    return text + ". Natural disasters cause damage to life and property. " \

            "Examples are earthquake, flood and cyclone."

resp1 = generate(original)

resp2 = generate(enriched)

print("Original Output:\n", resp1)

print("\nEnriched Output:\n", resp2)

print("\nWord Count:", len(resp1.split()), "vs", len(resp2.split()))
```

<u>Output</u>

Original Output:

write an essay on natural disaster. Natural disasters cause damage to life and property. Examples are earthquake, flood and cyclone.

Enriched Output:

write an essay on natural disaster including storm, damage, wind. Natural disasters cause damage to life and property. Examples are earthquake, flood and cyclone.

Word Count: 20 vs 24

Program Explained by line by line

1 Import Word2Vec

```
from gensim.models import Word2Vec
```

☞ Imports the **Word2Vec model** from the `gensim` library.

☞ Word2Vec is used to create **word embeddings** (numerical vector representations of words).

2 Create Training Data

```
data = [

["natural","disaster","earthquake","flood","cyclone","tsunami"
],
    ["earthquake","damage","buildings","people"],
    ["flood","rain","water","loss"],
    ["cyclone","storm","wind","destruction"]
]
```

☞ This is a **small dataset** created manually.

☞ Each inner list represents a sentence.

☞ These words are related to **natural disasters**.

☞ This data is used to train the Word2Vec model.

3 Train Word2Vec Model

```
model = Word2Vec(data, vector_size=50, min_count=1)
```

☞ Trains a Word2Vec model using the dataset.

Parameters:

- `data` → training sentences
- `vector_size=50` → each word is converted into a 50-dimensional vector
- `min_count=1` → include all words (even if they appear once)

☞ Now the model learns relationships between words.

4 Define Original Prompt

```
original = "write an essay on natural disaster"
```

☞ This is the **original input prompt** given to the AI.

5 Get Similar Words Using Embeddings

```
similar = [w for w, s in model.wv.most_similar("disaster",
topn=3)]
```

☞ `model.wv.most_similar("disaster")` finds words similar to “disaster”.

☞ `topn=3` → returns top 3 similar words.

☞ `w for w, s` → extracts only the word (ignores similarity score).

☞ This demonstrates the use of **word embeddings**.

Example result:

```
['storm', 'damage', 'wind']
```

6 Create Enriched Prompt

```
enriched = original + " including " + ", ".join(similar)
```

☞ Adds similar words to the original prompt.

☞ `", ".join(similar)` converts list into a sentence.

Result:

```
write an essay on natural disaster including storm, damage,
wind
```

☞ This is the **enriched prompt**.

7 Define Generate Function

```
def generate(text):
    return text + ". Natural disasters cause damage to life
and property. " \
           "Examples are earthquake, flood and cyclone."
```

☞ This is a **simple local text generator**.

☞ It adds fixed explanation text to the prompt.

☞ Used instead of real AI model (offline version).

8 Generate Outputs

```
resp1 = generate(original)
resp2 = generate(enriched)
```

☞ `resp1` → output using original prompt

☞ `resp2` → output using enriched prompt

9 Print Outputs

```
print("Original Output:\n", resp1)
```

☞ Prints output for original prompt.

```
print("\nEnriched Output:\n", resp2)
```

☞ Prints output for enriched prompt.

10 Compare Word Count

```
print("\nWord Count:", len(resp1.split()), "vs",
len(resp2.split()))
```

☞ `split()` divides sentence into words.

☞ `len()` counts number of words.

☞ Compares original vs enriched output length.

☞ Enriched output usually has more words → more detailed.

Viva Questions

1 What is Word2Vec?

Answer:

Word2Vec is a word embedding technique that converts words into numerical vectors based on their context in a dataset.

2 What are word embeddings?

Answer:

Word embeddings are vector representations of words where similar words have similar numerical values.

3 Why did you use Word2Vec in this program?

Answer:

We used Word2Vec to find semantically similar words to enrich the original prompt.

4 What is prompt enrichment?

Answer:

Prompt enrichment is the process of adding related or meaningful words to a prompt to improve the quality of AI-generated output.

5 What does `most_similar()` function do?

Answer:

It returns words that are semantically similar to a given word based on cosine similarity of vectors.

6 How does Word2Vec learn word relationships?

Answer:

Word2Vec uses neural networks (CBOW or Skip-gram) to predict context words. Words appearing in similar contexts get similar vector representations.

7 What is the difference between CBOW and Skip-gram?

Answer:

- **CBOW (Continuous Bag of Words):** Predicts a word using surrounding context words.
- **Skip-gram:** Predicts surrounding words using a target word. Skip-gram works better for small datasets.

8 What is cosine similarity in Word2Vec?

Answer:

Cosine similarity measures the similarity between two word vectors by calculating the cosine of the angle between them. Higher cosine value means higher similarity.

9 Why does enriched prompt give better output?

Answer:

Enriched prompts contain more contextual information, guiding the model to generate more detailed and relevant responses.

10 What is the role of `vector_size=50`?

Answer:

It defines the dimension of the word vector. Each word is represented as a 50-dimensional numerical vector.

11 What happens if `min_count=2` is used?

Answer:

Words that appear only once in the dataset will be ignored during training.

12 Can Word2Vec understand word meaning?

Answer:

Word2Vec does not truly understand meaning. It captures statistical relationships based on word co-occurrence.

13 How does this concept help Generative AI models?

Answer:

By enriching prompts using semantically similar words, we provide more context, improving the relevance and depth of generated responses.

14 What are limitations of Word2Vec?

Answer:

- Cannot handle out-of-vocabulary words
- Context-independent (same vector for a word in all contexts)
- Requires sufficient training data

15 How is Word2Vec different from modern embeddings like BERT?

Answer:

Word2Vec generates static embeddings (same vector for every context). BERT generates contextual embeddings (word meaning changes based on sentence context).